

CENTRO UNIVERSITÁRIO INTERNACIONAL UNINTER
ESCOLA SUPERIOR POLITÉCNICA
ANÁLISE E DESENVOLVIMENTO DE SISTEMAS
DESENVOLVIMENTO WEB BACK END

ATIVIDADE PRÁTICA
API REST – BAOZI STORE

JULIA SAMPAIO – RU: 4913744

Braga – Portugal

2026

ATIVIDADE PRÁTICA

1. Descrição de uma situação fictícia

A **Baozi Store** é uma pequena loja que vende **pão chinês**. Para melhorar a organização do negócio, foi criado um sistema simples para controlar clientes, produtos e pedidos.

Um cliente chamado **JuliaSampaio4913744** realizou seu cadastro no sistema. O produto vendido pela loja chama-se **Baozi de Carne** e é vendido por unidade, ao preço de 4.50. Em um determinado momento, o cliente realizou um pedido de **5** unidades do produto.

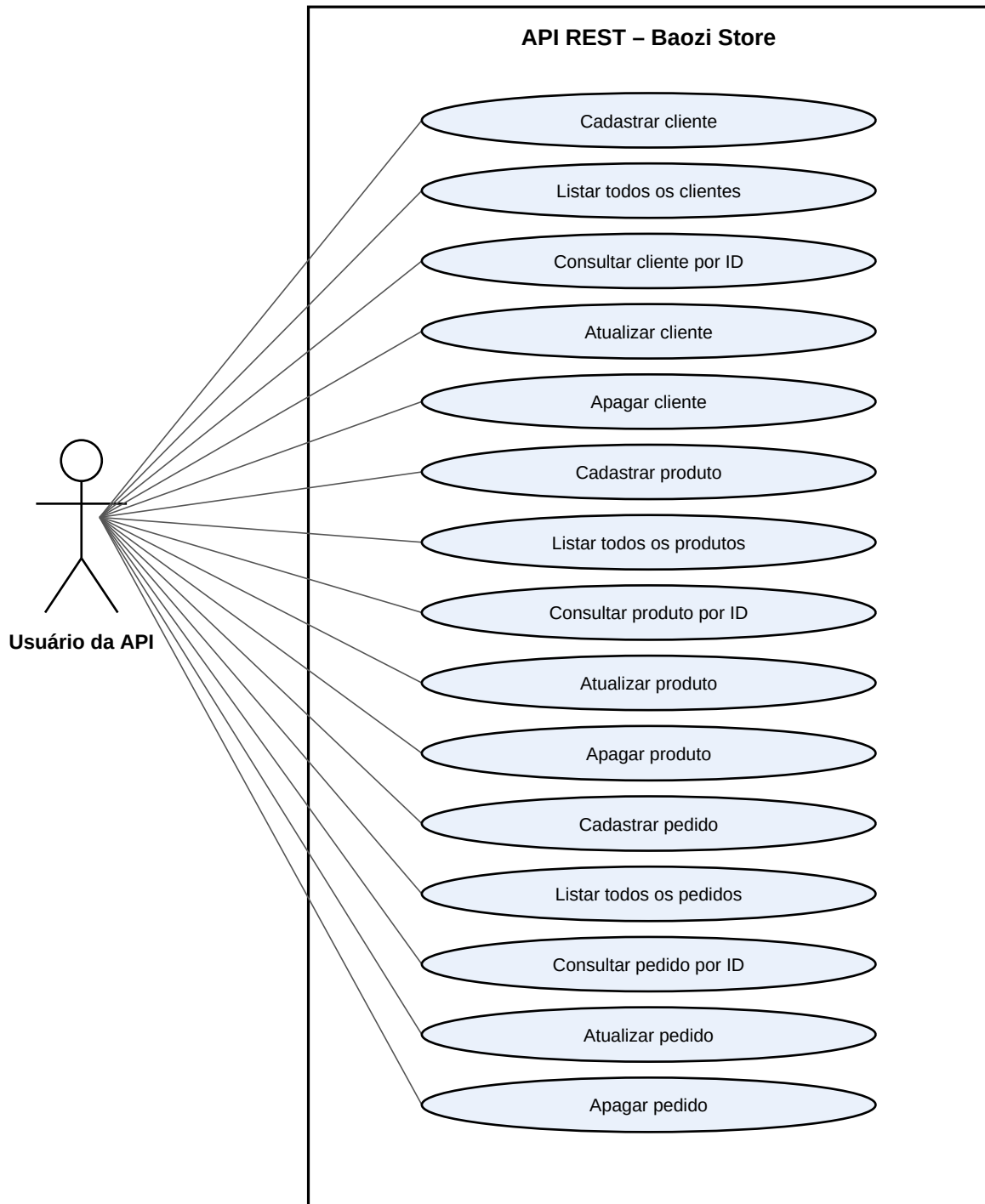
O sistema registra o cliente, o produto comprado e a quantidade solicitada, facilitando o controle da loja.

Dados utilizados nos testes (presentes nos prints do Postman):

Entidade	Dados
Cliente	nome: JuliaSampaio4913744
Produto	nome: Baozi de Carne; preco: 4.50; estoque: true
Pedido	cliente: JuliaSampaio4913744 (id 1); produto: Baozi de Carne (id 1); quantidade: 5

2. Diagrama caso de uso

Diagrama de Caso de Uso UML com o ator principal (Usuário da API) e os casos de uso do CRUD de Cliente, Produto e Pedido implementados na API REST.



3. Especificação da API desenvolvida

Tecnologias: Java 17, Spring Boot, Spring Data JPA, banco relacional H2 (perfil alternativo MySQL), JSON nos endpoints, padrão MVC do Spring. Pacotes: *model*, *repository*, *controller* (e *exception* para tratamento de erros).

3.1 Entidades criadas e campos

Entidade	Campo	Tipo	Regras
Cliente	id	Long	Chave primária, gerada automaticamente
	nome	String	Obrigatório, até 100 caracteres
	clienteDesde	LocalDate	Não pode ser data futura; padrão: data atual
Produto	id	Long	Chave primária, gerada automaticamente
	nome	String	Obrigatório, até 100 caracteres
	preco	BigDecimal	Obrigatório, maior que zero, 2 casas decimais
	estoque	Boolean	Obrigatório (true = disponível)
Pedido	id	Long	Chave primária, gerada automaticamente
	clienteId	Long	Obrigatório; deve referenciar cliente existente
	produtoId	Long	Obrigatório; deve referenciar produto existente com estoque
	quantidade	Integer	Obrigatório, entre 1 e 1000

3.2 Endpoints implementados

Método	Rota	Descrição	Resposta
POST	/clientes	Criar cliente	201 Created
GET	/clientes	Listar todos	200 OK
GET	/clientes/{id}	Consultar por ID	200 OK / 404
PUT	/clientes/{id}	Atualizar (opcional)	200 OK / 404
DELETE	/clientes/{id}	Apagar	204 No Content / 404 / 409
POST	/produtos	Criar produto	201 Created
GET	/produtos	Listar todos	200 OK
GET	/produtos/{id}	Consultar por ID	200 OK / 404
PUT	/produtos/{id}	Atualizar (opcional)	200 OK / 404
DELETE	/produtos/{id}	Apagar	204 No Content / 404 / 409
POST	/pedidos	Criar pedido	201 Created
GET	/pedidos	Listar todos	200 OK
GET	/pedidos/{id}	Consultar por ID	200 OK / 404
PUT	/pedidos/{id}	Atualizar (opcional)	200 OK / 404

Método	Rota	Descrição	Resposta
DELETE	/pedidos/{id}	Apagar	204 No Content / 404 / 409

3.3 Boas práticas de segurança aplicadas

Controle	Implementação
Validação de entrada	Bean Validation (@Valid, @NotBlank, @Positive, @Min/@Max) com resposta 400 e mensagens por campo
Mass assignment	O id é sempre ignorado no POST; o banco o gera
Integridade referencial	Pedido só é aceito com cliente e produto existentes e produto em estoque; cliente/produto com pedidos não podem ser apagados (409)
Tratamento de erros	Handler global devolve mensagens genéricas; stack trace, SQL e nomes de classes ficam apenas no log do servidor
Gestão de credenciais	Banco MySQL configurado por variáveis de ambiente; nenhuma senha no repositório
Redução de superfície	Console H2 desativado; JSON como único formato de troca

4. Prints do Postman

Prints obrigatórios, cada um com as informações da descrição (cliente, produto e quantidade) visíveis.

4.1 POST /clientes

Body (raw JSON):

```
{"nome": "JuliaSampaio4913744"}
```

[INSERIR AQUI O PRINT DO POSTMAN]

4.2 POST /produtos

Body (raw JSON):

```
{"nome": "Baozi de Carne", "preco": 4.5, "estoque": true}
```

[INSERIR AQUI O PRINT DO POSTMAN]

4.3 POST /pedidos

Body (raw JSON):

```
{"clienteId": 1, "produtoId": 1, "quantidade": 5}
```

[INSERIR AQUI O PRINT DO POSTMAN]

4.4 GET geral

GET /clientes, GET /produtos e GET /pedidos (um print para cada ou um por rota)

GET http://localhost:8080/clientes

[INSERIR AQUI O PRINT DO POSTMAN]

4.5 GET por ID

GET /pedidos/1 (e, se desejar, /clientes/1 e /produtos/1).
GET http://localhost:8080/pedidos/1

[INSERIR AQUI O PRINT DO POSTMAN]

4.6 DELETE

DELETE /pedidos/1 retorna 204. Em seguida, GET /pedidos/1 retorna 404.
DELETE http://localhost:8080/pedidos/1

[INSERIR AQUI O PRINT DO POSTMAN]

5. Link do repositório (GitHub ou similar) contendo o projeto (código fonte)

https://github.com/SEU_USUARIO/baozi-store